

UNIT IV

DATA STORAGE

4.1 FLAT FILE AND ITS ADVANTAGES

Data

- Data is a set of facts such as descriptions, observations, and numbers used in decision making.
- We can classify data as structured, unstructured, or semi-structured data.

Structured data

- Structured data is generally tabular data that is represented by columns and rows in a database.
- Databases that hold tables in this form are called relational databases.
- The mathematical term “relation” specifies to a formed set of data held as a table.
- In structured data, all row in a table has the same set of columns.

Semi structured Data

- Semi-structured data is information that doesn't consist of structured data (relational database) but still has some structure to it.

Unstructured Data

- Unstructured data is information that either does not organize in a pre-defined manner or not have a pre-defined data model.
- Unstructured information is a set of text-heavy but may contain data such as numbers, dates, and facts as well.
- Videos, audio, and binary data files might not have a specific structure. They're assigned to as unstructured data.

Need for Database

- A database, in the most general sense, is an organized collection of data. More specifically, a database is an electronic system that allows data to be easily accessed, manipulated and updated.
- In other words, a database is used by an organization as an electronic way to store, manage and retrieve information. The database is one of the cornerstones of enterprise IT, and its ability to organize, process and manage information in a structured and controlled manner is the key to many aspects of modern business efficiency.

Flat File Database

A flat file database is usually one table with rows of information. An example of this can be a spreadsheet of a list of people containing their names, surnames, ID numbers, and dates of birth. There is a row for every person; in this case, no person will have more than one row, as one can only have one ID number and one date of birth.

Example: The contact list of a smartphone is also a good example of a flat-file database.

What are the key characteristics of a flat-file database?

1. Single Table Structure

- **Description:** A flat-file database consists of a single table that holds all the data. Each row in the table represents a record, and each column represents a field or attribute of that record.
- **Example:** A contact list database where each row contains a person's name, phone number, and email address.

2. No Data Relationships

- **Description:** Unlike relational databases, a flat-file database does not store relationships between tables. All data is stored in a single table without links or references to other tables.
- **Example:** A flat-file database might store all employee information in one table rather than separating employee details and department information into separate tables with relationships.

3. Simple Data Format

- **Description:** Data is often stored in simple formats like plain text or CSV (comma-separated values), which makes it easy to read and edit manually if needed.

Example: A CSV file with rows of data like.:

Name, Age, Email

John Doe, 30, john@example.com

Jane Smith, 25, jane@example.com

4. No Complex Querying

- **Description:** Flat-file databases generally don't support complex querying capabilities (like SQL in relational databases). Queries tend to be simpler and usually involve searching through the entire table or filtering by specific columns.
- **Example:** Searching for all records where the "Age" field is greater than 25.

5. Limited Scalability

- **Description:** Flat-file databases are ideal for small datasets but can become inefficient as the dataset grows. They don't handle large amounts of data or complex data relationships as efficiently as relational databases.
- **Example:** A flat-file database might work well for a small contact list, but performance can degrade as the number of records grows into thousands or more.

6. No Data Integrity Constraints

- **Description:** Flat-file databases typically do not enforce rules for data integrity, such as unique constraints or foreign key relationships. Data entry is more prone to errors without these protections.
- **Example:** You might accidentally enter a duplicate record for a contact without any system alert.

7. Easy to Use and Set Up

- **Description:** Flat-file databases are simple to set up and don't require advanced technical knowledge. They are often managed via basic tools like text editors or spreadsheet software.
- **Example:** Creating and maintaining a simple contact list in a spreadsheet or a CSV file.

8. Human-Readable Format

- **Description:** Data is usually stored in a human-readable format, making it easy to understand and edit manually. This characteristic makes flat-file databases accessible for non-technical users.
- **Example:** A text editor can be used to open and edit a CSV file containing customer data.

9. Minimal Overhead

- **Description:** Flat-file databases have low overhead since they don't require complex systems or additional software. They can be used for basic data management without the need for a full database management system.
- **Example:** Storing data in a simple text file or CSV file without requiring a complex database server.

10. Portability

- **Description:** Flat-file databases are highly portable because they are stored in simple file formats. The data can easily be transferred between systems or backed up.
- **Example:** A CSV file containing product information can be moved from one computer to another without compatibility issues.

What is a flat-file database used for?

A **flat-file database** is used for storing data in a simple, single-table format. It is particularly suitable for smaller datasets and situations where the data structure does not require complex relationships or advanced querying capabilities. Here are some common use cases:

1. Simple Data Storage

- **Example:** Storing lists of information like names, addresses, phone numbers, and email addresses.
- **Use Case:** A personal contact list or a small company's employee directory.

2. Inventory Management

- **Example:** Tracking product details such as product names, quantities, prices, and SKU numbers.
- **Use Case:** Small businesses or retail shops using a simple database to track inventory without needing complex relationships between products and suppliers.

3. Address Books

- **Example:** Storing contact information like names, addresses, phone numbers, and email addresses.
- **Use Case:** Keeping a personal or professional address book for easy reference.

4. Task or Project Management

- **Example:** Tracking tasks, deadlines, and responsible persons.
- **Use Case:** A project manager might use a flat-file database to keep track of tasks in a project, such as task name, due date, and status.

5. Financial Tracking

- **Example:** Recording expenses, income, or small budgets.
- **Use Case:** A freelance worker or a small business owner can track their financial transactions in a simple flat-file database.

6. Personal Collection Management

- **Example:** Cataloging items in a personal collection like books, movies, music, or stamps.
- **Use Case:** A person might use a flat-file database to catalog their personal library or DVD collection.

7. Small-Scale Data Analysis

- **Example:** Storing small sets of data for basic analysis (e.g., survey results, customer feedback).

- **Use Case:** Collecting and analyzing survey responses or customer feedback without needing a full relational database system.

8. Logging and Event Tracking

- **Example:** Storing logs, such as system events or transaction logs.
- **Use Case:** Logging system activity or tracking user interactions with a website.

9. Recipe and Cookbook Management

- **Example:** Storing recipes with ingredients, cooking times, and instructions.
- **Use Case:** A home cook or a small business might use a flat-file database to store and organize recipes for easy access.

10. Data Import and Export

- **Example:** Importing and exporting data between systems in CSV format for analysis or processing.
- **Use Case:** Transferring data from one system to another in a format that is easy to read, process, and understand.

11. Simple Client or Customer Databases

- **Example:** Storing basic client or customer information (name, email, phone).
- **Use Case:** Freelancers or small businesses may use flat-file databases to track client details without needing advanced CRM (Customer Relationship Management) software.

When to Use a Flat-File Database

- **Small-scale applications:** Where the data is not large enough to justify the complexity of a relational database system.
- **Non-technical users:** For individuals who need to store and manage data without advanced technical knowledge.
- **Portability:** When you need the data to be in a simple, easily portable format (e.g., CSV, text).
- **Quick setup and use:** When you need a simple solution for data entry and retrieval without much setup or maintenance.

Advantages of Flat File Database:

- **Simplicity:** Easy to understand and set up, making it ideal for small-scale or straightforward data storage needs.
- **Low Resource Requirement:** Requires minimal system resources compared to more complex database systems.

- No Need for Specialized Software: Can be managed using common tools like text editors, spreadsheets, or basic database software.
- Ease of Portability: Flat file databases are typically stored in simple formats (like CSV or text files), making them easy to transfer between systems.
- Flexible Formats: Data can be stored in various formats (e.g., plain text, CSV, TSV) depending on the user's needs.
- Direct Access: Data can often be edited directly without the need for complex interfaces or database management systems.
- Ideal for Small-Scale Applications: Well-suited for managing small datasets such as contact lists, inventory, or single-table data structures.
- Wide Compatibility: Flat file formats like CSV are widely supported by different software tools, including databases, programming languages, and spreadsheet software.
- Cost-Effective: Does not require expensive software or licenses, especially when using free tools like text editors or spreadsheet programs.
- Quick Setup and Use: Minimal setup time is required, making it efficient for quick data management tasks.
- Transparency: Data is stored in a human-readable format (e.g., plain text), making it easy to understand and debug.
- No Overhead of Relationships: Ideal for situations where relationships between tables are not required, reducing complexity.

Disadvantages of flat-file database

A list of disadvantages of a flat-file database is given below:

- Flat file databases are harder to update.
- Harder to change data format.
- It is a poor database in terms of complex queries.
- It increased Redundancy and inconsistency.

The below table shows the difference between Flat file database and relational database -

Flat file database	Relational database
Flat file database is used to store data in a single table structure.	Relational database is used to store data in a multiple-table structure.
Flat file databases can be accessed by a variety of software applications.	Relational database uses a Relational Database Management System (RDBMS) to access the data.

Flat file database includes software like FileMaker, Berkeley DB, and Borland Reflex.	Relational database includes software like Oracle, MySQL, and PostgreSQL.
It is simple, portable, easy to use, and inexpensive.	It is more powerful and more efficient than the Flat file database.
It is represented using a Data dictionary.	It is represented using a Schema.
It contains files, records, characters, and fields.	It contains the entity's attributes and relationships.
In a Flat file database, there is a problem of data redundancy.	In a Relational database, there is no problem with data redundancy.
It is less secure.	It is more secure than the Flat file database.
Mostly, small organizations use the flat file database.	Mostly, large organizations use a relational database.

4.2 RELATIONAL DATABASES AND ITS ADVANTAGES

What is a Database Table?

- A table is a collection of related data entries, and it consists of columns and rows.
- A column holds specific information about every record in the table.
- A record (or row) is each individual entry that exists in a table.
- Look at a selection from the Northwind "Customers" table:

The columns in the "Customers" table above are: CustomerID, CustomerName, ContactName, Address, City, PostalCode and Country. The table has 5 records (rows).

Customer ID	CustomerName	ContactName	Address	Postal Code	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	12209	Germany
2	Ana Trujillo Emparedadosy helados	Ana Trujillo	Avda. de la Constitución 2222	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	S-958 22	Sweden

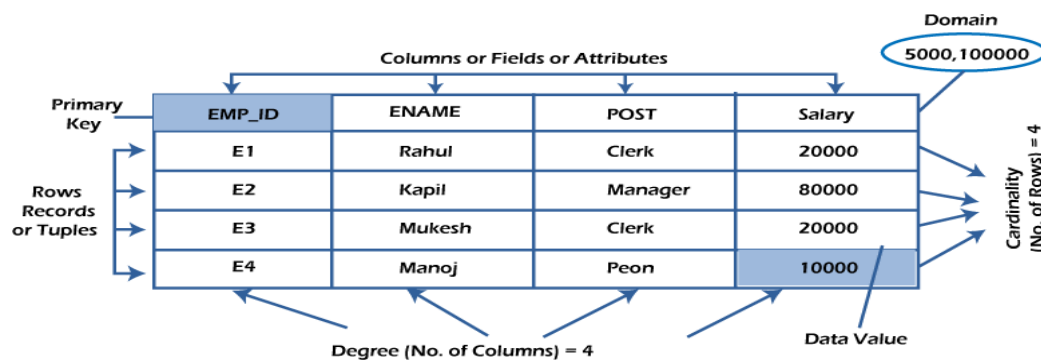
What is RDBMS (Relational Database Management System)

RDBMS stands for Relational Database Management System. All modern database management systems like SQL, MS SQL Server, IBM DB2, ORACLE, My-SQL, and Microsoft Access are based on RDBMS. It is called Relational Database Management System (RDBMS) because it is based on the relational model introduced by E.F. Codd.

How it works

Data is represented in terms of tuples (rows) in RDBMS. A relational database is the most commonly used database. It contains several tables, and each table has its primary key. Due to a collection of an organized set of tables, data can be accessed easily in RDBMS.

Following are the various terminologies of RDBMS:



What is table/Relation?

Everything in a relational database is stored in the form of relations. The RDBMS database uses tables to store data. A table is a collection of related data entries and contains rows and columns to store data. Each table represents some real-world objects such as person, place, or event about which information is collected. The organized collection of data into a relational table is known as the logical view of the database.

Properties of a Relation:

- Each relation has a unique name by which it is identified in the database.
- Relation does not contain duplicate tuples.
- The tuples of a relation have no specific order.
- All attributes in a relation are atomic, i.e., each cell of a relation contains exactly one value.

A table is the simplest example of data stored in RDBMS.

Let's see the example of the student table.

ID	Name	AGE	COURSE
1	Ajeet	24	B.Tech
2	aryan	20	C.A
3	Mahesh	21	BCA
4	Ratan	22	MCA
5	Vimal	26	BSC

What is a row or record?

A row of a table is also called a record or tuple. It contains the specific information of each entry in the table. It is a horizontal entity in the table. For example, The above table contains 5 records.

Properties of a row:

- No two tuples are identical to each other in all their entries.
- All tuples of the relation have the same format and the same number of entries.
- The order of the tuple is irrelevant. They are identified by their content, not by their position.

Let's see one record/row in the table.

ID	Name	AGE	COURSE
1	Ajeet	24	B.Tech

What is a column/attribute?

A column is a vertical entity in the table which contains all information associated with a specific field in a table. For example, "name" is a column in the above table which contains all information about a student's name.

Properties of an Attribute:

- Every attribute of a relation must have a name.
- Null values are permitted for the attributes.
- Default values can be specified for an attribute automatically inserted if no other value is specified for an attribute.
- Attributes that uniquely identify each tuple of a relation are the primary key.

Name
Ajeet
Aryan
Mahesh

What is data item/Cells?

The smallest unit of data in the table is the individual data item. It is stored at the intersection of tuples and attributes.

Properties of data items:

- Data items are atomic.
- The data items for an attribute should be drawn from the same domain.

In the below example, the data item in the student table consists of Ajeet, 24 and Btech, etc.

ID	Name	AGE	COURSE
1	Ajeet	24	B.Tech

Degree:

The total number of attributes that comprise a relation is known as the degree of the table.

For example, the student table has 4 attributes, and its degree is 4.

ID	Name	AGE	COURSE
1	Ajeet	24	B.Tech
2	aryan	20	C.A
3	Mahesh	21	BCA
4	Ratan	22	MCA
5	Vimal	26	BSC

Cardinality:

The total number of tuples at any one time in a relation is known as the table's cardinality. The relation whose cardinality is 0 is called an empty table.

For example, the student table has 2 rows, and its cardinality is 2.

ID	Name	AGE	COURSE
1	Ajeet	24	B.Tech
2	aryan	20	C.A

Domain:

The domain refers to the possible values each attribute can contain. It can be specified using standard data types such as integers, floating numbers, etc. For example, An attribute entitled Marital_Status may be limited to married or unmarried values.

NULL Values

The NULL value of the table specifies that the field has been left blank during record creation. It is different from the value filled with zero or a field that contains space.

Data Integrity: There are the following categories of data integrity exist with each RDBMS:

Entity integrity: It specifies that there should be no duplicate rows in a table.

Domain integrity: It enforces valid entries for a given column by restricting the type, the format, or the range of values.

Referential integrity specifies that rows cannot be deleted, which are used by other records.

User-defined integrity: It enforces some specific business rules defined by users. These rules are different from the entity, domain, or referential integrity.

Relational Model Notation

- Relation schema R of degree n is denoted by $R(A_1, A_2, \dots, A_n)$.
- Uppercase letters Q, R, S denote relation names.
- Lowercase letters q, r, s denote relation states.
- Letters t, u, v denote tuples.
- In general, the name of a relation schema such as STUDENT also indicates the current set of tuples in that relation.
- An attribute A can be qualified with the relation name R to which it belongs by using the dot notation R.A for example, STUDENT.Name or STUDENT.Age.
- An n-tuple t in a relation r(R) is represented as $t = \langle v_1, v_2, \dots, v_n \rangle$ where v_i is the value corresponding to the attribute A_i . The value v_i for attribute A_i in tuple t can be accessed using $t[A_i]$ or $t.A_i$.

Characteristics of the Relational Model

1. **Data Representation:** Data is organized in tables (relations), with rows (tuples) representing records and columns (attributes) representing data fields.
2. **Atomic Values:** Each attribute in a table contains atomic values, meaning no multi-valued or nested data is allowed in a single cell.
3. **Unique Keys:** Every table has a primary key to uniquely identify each record, ensuring no duplicate rows.
4. **Attribute Domain:** Each attribute has a defined domain, specifying the valid data types and constraints for the values it can hold.
5. **Tuples as Rows:** Rows in a table, called tuples, represent individual records or instances of real-world entities or relationships.
6. **Relation Schema:** A table's structure is defined by its schema, which specifies the table name, attributes, and their domains.
7. **Data Independence:** The model ensures logical and physical data independence, allowing changes in the database schema without affecting the application layer.
8. **Integrity Constraints:** The model enforces rules like:
9. **Domain constraints:** Attribute values must match the specified domain.
10. **Entity integrity:** No primary key can have NULL values.
11. **Referential integrity:** Foreign keys must match primary keys in the referenced table or be NULL.
12. **Relational Operations:** Supports operations like **selection, projection, join, union, and intersection**, enabling powerful data retrieval and manipulation.
13. **Data Consistency:** Ensures data consistency through constraints, reducing redundancy and anomalies.
14. **Set-Based Representation:** Tables in the relational model are treated as sets, and operations follow mathematical set theory principles.

Constraints in Relational Model

While designing the Relational Model, we define some conditions which must hold for data present in the database are called Constraints. These constraints are checked before performing any operation (insertion, deletion, and updation) in the database. If there is a violation of any of the constraints, the operation will fail.

Domain Constraints

Domain Constraints ensure that the value of each attribute A in a tuple must be an atomic value derived from its specified domain, $\text{dom}(A)$. Domains are defined by the data types associated with the attributes. Common data types include:

- **Numeric types:** Integers (e.g., short integer, integer, long integer) and real numbers (e.g., float, double-precision float).
- **Character types:** Fixed-length and variable-length strings.
- **Boolean values.**
- **Specialized types:** Date, time, timestamp, monetary values, and others.

Key Integrity

Every relation in the database should have at least one set of attributes that defines a tuple uniquely. Those set of attributes is called keys. e.g.; ROLL_NO in STUDENT is key. No two students can have the same roll number. So a key has two properties:

- It should be unique for all tuples.
- It can't have NULL values.

Relational Model Operations:

The relational model provides a rich set of operations that allow for powerful manipulation and retrieval of data stored in relational databases. These operations are fundamental to interacting with the data in relational databases, such as MySQL, PostgreSQL, or Oracle. Below are the key **relational operations** in the relational model:

1. Selection (σ)

Definition: The **selection** operation is used to retrieve rows (tuples) from a relation (table) that satisfy a specific condition. It is also referred to as **restriction**. In simple terms, it filters the rows based on a condition (predicate).

Syntax:

$\sigma_{\text{condition}}(\text{Relation})$

where condition is the condition that must be satisfied by the rows, and Relation is the name of the table.

Example: Consider the STUDENT table:

ROLL_NO	NAME	AGE	BRANCH_CODE
1	RAM	18	CS
2	RAMESH	18	CS
3	SUJIT	20	ECE
4	SURESH	22	IT

To select students who are 18 years old, the **selection** operation would be:

$\sigma_{\text{AGE}=18}(\text{STUDENT})$

The result will be:

ROLL_NO	NAME	AGE	BRANCH_CODE
1	RAM	18	CS
2	RAMESH	18	CS

2. Projection (π)

Definition: The **projection** operation is used to retrieve specific columns (attributes) from a relation, essentially removing unnecessary columns. It reduces the number of attributes in the result.

Syntax:

$\pi_{\text{attribute_list}}(\text{Relation})$

where attribute_list is the list of attributes (columns) to be retrieved, and Relation is the name of the table.

Example: From the STUDENT table, if you want to retrieve only the NAME and AGE columns, the **projection** operation would be:

$\pi_{\text{NAME, AGE}}(\text{STUDENT})$

The result will be:

NAME	AGE
RAM	18
RAMESH	18
SUJIT	20
SURESH	22

3. Join ($\bowtie$)

Definition: The **join** operation combines rows from two or more tables based on a related column (usually a foreign key relationship). The most common type of join is the **inner join**, but there are several types, including left join, right join, and outer join.

Syntax:

Relation1 $\bowtie$ Relation2

where Relation1 and Relation2 are two relations (tables) that are being joined.

Types of Joins:

1. **Inner Join:** Retrieves only the rows that have matching values in both tables.

2. **Left Outer Join:** Retrieves all rows from the left table and the matching rows from the right table. If no match is found, NULL values are returned for columns from the right table.
3. **Right Outer Join:** Similar to left outer join, but retrieves all rows from the right table and matching rows from the left.
4. **Full Outer Join:** Retrieves all rows from both tables, and fills in NULL for non-matching rows.

Example: Consider two tables, STUDENT and BRANCH:

STUDENT Table:

ROLL_NO	NAME	BRANCH_CODE
1	RAM	CS
2	RAMESH	CS
3	SUJIT	ECE

BRANCH Table:

BRANCH_CODE	BRANCH_NAME
CS	Computer Science
ECE	Electronics and Communication Engineering

To perform an **inner join** between STUDENT and BRANCH on the BRANCH_CODE column, the query would be:

STUDENT $\bowtie$ BRANCH

Result:

ROLL NO	NAME	BRANCH_CODE	BRANCH_NAME
1	RAM	CS	Computer Science
2	RAMESH	CS	Computer Science
3	SUJIT	ECE	Electronics and Communication Engineering

Notice how the data from both tables is combined based on the BRANCH_CODE.

4. Union ($\cup$)

Definition: The **union** operation combines the results of two queries into one, excluding any duplicate rows. Both queries must have the same number of columns, and corresponding columns must have compatible data types.

Syntax:

Relation1 $\cup$ Relation2

Example: Consider two tables:

STUDENT_1 Table:

ROLL_NO	NAME	AGE
1	RAM	18
2	RAMESH	18

STUDENT_2 Table:

ROLL_NO	NAME	AGE
3	SUJIT	20
4	SURESH	22

Performing a **union** between these two tables:

STUDENT_1 $\cup$ STUDENT_2

The result will be:

ROLL_NO	NAME	AGE
1	RAM	18
2	RAMESH	18
3	SUJIT	20
4	SURESH	22

Notice that if there were any duplicates between the two sets, they would have been removed.

5. Intersection ($\cap$)

Definition: The **intersection** operation returns only the rows that appear in both of the input relations. Like the union operation, both tables must have the same number of columns, and the data types must match.

Syntax:

$$\text{Relation1} \cap \text{Relation2}$$

Example: Consider two tables, STUDENT_A and STUDENT_B, both with the same columns (ROLL_NO, NAME, and AGE).

STUDENT_A Table:

ROLL_NO	NAME	AGE
1	RAM	18
2	RAMESH	18

STUDENT_B Table:

ROLL_NO	NAME	AGE
2	RAMESH	18
3	SUJIT	20

The **intersection** of STUDENT_A and STUDENT_B would be:

$$\text{STUDENT_A} \cap \text{STUDENT_B}$$

Result:

ROLL_NO	NAME	AGE
2	RAMESH	18

6. Difference (–)

Definition: The **difference** operation returns the rows that appear in the first relation but not in the second. It is sometimes referred to as **set difference** or **minus**.

Syntax:

$$\text{Relation1} - \text{Relation2}$$

Example: Continuing with the previous tables, performing a **difference** operation on STUDENT_A and STUDENT_B: $\text{STUDENT_A} - \text{STUDENT_B}$

Result:

ROLL_NO	NAME	AGE
1	RAM	18

4.2.1 What is a Relational Database?

A relational database defines database relationships in the form of tables. The tables are related to each other - based on data common to each.

Look at the following three tables "Customers", "Orders", and "Shippers" from the Northwind database:

Customers Table

Custo merID	Customer Name	ContactName	Address	Postal Code	Country
1	Alfreds Futterkiste	Maria Anders	Obere Str. 57	12209	Germany
2	Ana Trujillo Emparedadoy helados	Ana Trujillo	Avda. de la Constitución 2222	05021	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mataderos 2312	05023	Mexico
4	Around the Horn	Thomas Hardy	120 Hanover Sq.	WA1 1DP	UK
5	Berglunds snabbköp	Christina Berglund	Berguvsvägen 8	S-958 22	Sweden

The relationship between the "Customers" table and the "Orders" table is the CustomerID column:

Orders Table

OrderID	CustomerID	EmployeeID	OrderDate	ShipperID
10278	5	8	1996-08-12	2
10280	5	2	1996-08-14	1
10308	2	7	1996-09-18	3
10355	4	6	1996-11-15	1
10365	3	3	1996-11-27	2
10383	4	8	1996-12-16	3
10384	5	3	1996-12-16	3

The relationship between the "Orders" table and the "Shippers" table is the ShipperID column:

Shippers Table

ShipperID	ShipperName	Phone
1	Speedy Express	(503) 555-9831
2	United Package	(503) 555-3199
3	Federal Shipping	(503) 555-9931

Difference between DBMS and RDBMS

Although DBMS and RDBMS both are used to store information in physical database but there are some remarkable differences between them.

The main differences between DBMS and RDBMS are given below:

No	DBMS	RDBMS
1)	DBMS applications store data as file.	RDBMS applications store data in a tabular form.
2)	In DBMS, data is generally stored in either a hierarchical form or a navigational form.	In RDBMS, the tables have an identifier called primary key and the data values are stored in the form of tables.
3)	Normalization is not present in DBMS.	Normalization is present in RDBMS.
4)	DBMS does not apply any security with regards to data manipulation.	RDBMS defines the integrity constraint for the purpose of ACID (Atomocity, Consistency, Isolation and Durability) property.
5)	DBMS uses file system to store data, so there will be no relation between the tables.	RDBMS, data values are stored in the form of tables, so a relationship between these data values will be stored in the form of a table as well.
6)	DBMS has to provide some uniform methods to access the stored information.	RDBMS system supports a tabular structure of the data and a relationship between them to access the stored information.
7)	DBMS does not support distributed databases.	RDBMS supports distributed databases.

8)	DBMS is meant for small organizations and deals with small data. It supports single users.	RDBMS is designed to handle large amounts of data. it supports multiple users.
9)	Examples of DBMS are file systems, xml etc.	Examples of RDBMS are mysql, postgre, sql server, oracle etc.

After observing the differences between DBMS and RDBMS, you can say that RDBMS is an extension of DBMS. There are many software products in the market today who are compatible with both DBMS and RDBMS. Means today a RDBMS application is a DBMS application and vice-versa.

Benefits of relational databases

The main benefit of the relational database model is that it provides an intuitive way to represent data and allows easy access to related data points. As a result, relational databases are most commonly used by organizations that need to manage large amounts of structured data, from tracking inventory to processing transactional data to application logging.

There are many other advantages to using relational databases to manage and store your data, including:

Flexibility

It's easy to add, update, or delete tables, relationships, and make other changes to data whenever you need without changing the overall database structure or impacting existing applications.

ACID compliance

Relational databases support ACID (Atomicity, Consistency, Isolation, Durability) performance to ensure data validity regardless of errors, failures, or other potential mishaps.

Ease of use

It's easy to run complex queries using SQL, which enables even non-technical users to learn how to interact with the database.

Collaboration

Multiple people can operate and access data simultaneously. Built-in locking prevents simultaneous access to data when it's being updated.

Built-in security

Role-based security ensures data access is limited to specific users.

Database normalization

Relational databases employ a design technique known as normalization that reduces data redundancy and improves data integrity.

4.3 DATA READ & WRITE IN LOCAL STORAGE

Local storage refers to the storage space available on a device's hard drive, solid-state drive (SSD), or other physical storage mediums. It is a non-volatile storage system, meaning that data stored in local storage remains persistent even when the device is powered off. Local storage is a key component of computing and is widely used for personal, professional, and enterprise purposes. This document explores the concepts, mechanisms, advantages, and limitations of data read and write operations in local storage.

1. Overview of Local Storage

Local storage involves storing data directly on a device, making it accessible without relying on a network or external servers. The data can include files, documents, media, application data, and more. It is commonly used in personal computers, laptops, and mobile devices for various purposes such as:

- Storing operating systems and applications.
- Saving user-generated content, such as documents and photos.
- Caching data for offline access.

Key Characteristics of Local Storage:

- **Persistence:** Data remains stored even after the device is powered off.
- **Accessibility:** Data can be accessed and managed through the device's file system.
- **Performance:** Offers high-speed data read and write operations with minimal latency.

2. Mechanisms of Data Read & Write Local storage operations rely on the underlying file system of the device's operating system. The processes for reading and writing data involve interacting with files and directories through programming or manually using file explorers. Below are the mechanisms involved:

Writing Data to Local Storage:

1. Opening a File or Creating a New File:

- A file is opened in write mode or a new file is created.

Example (Python):

with open('example.txt', 'w') as file:

- file.write('Hello, Local Storage!')

2. Writing Data:

- Data is written to the file in the desired format (text, binary, etc.).
- For large-scale storage, applications may use block-level storage for efficiency.

3. Closing the File:

- Ensures that all data is saved and resources are freed.

Reading Data from Local Storage:

1. Opening the File:

- A file is opened in read mode to access its contents.

Example (Python):

```
with open('example.txt', 'r') as file:  
    content = file.read()  
  
print(content)
```

2. Reading Data:

- The content of the file is read sequentially or using specific access patterns.
- Applications may use buffering to optimize large file reads.

3. Processing the Data:

- The retrieved data is converted or deserialized into a format usable by the application.

Advantages of Local Storage

Local storage has several benefits, making it a preferred choice for many applications. Below are its key advantages:

1. High Performance:

- Local storage provides fast read and write speeds with low latency.
- Ideal for applications requiring rapid data access, such as gaming or video editing.

2. Privacy and Security:

- Data stored locally is not accessible to other users or devices unless explicitly shared.
- Offers enhanced security for sensitive data compared to cloud storage.

3. Cost-Effectiveness:

- Local storage involves a one-time hardware investment.
- No recurring costs for data storage or bandwidth usage.

4. Offline Accessibility:

- Data stored locally can be accessed without an internet connection.
- Essential for applications or environments with limited connectivity.

Limitations of Local Storage

Despite its advantages, local storage has certain limitations that may affect its suitability for specific use cases:

- **Limited Storage Capacity:** Storage devices have finite space, which may not be sufficient for large-scale applications.
- **Hardware Dependency:** Local storage is tied to the physical device, making it inaccessible if the device is damaged or lost.
- **Lack of Remote Accessibility:** Unlike cloud storage, local storage does not support easy data sharing or remote access.
- **Risk of Hardware Failure:** Devices such as hard drives and SSDs are susceptible to failure, leading to potential data loss.

Types of Computer Memory

1. Primary Memory
2. Secondary Memory
3. Tertiary Memory

1. Primary Memory: It is also known as internal memory and main memory. This is a section of the CPU that holds program instructions, input data, and intermediate results. It is generally smaller in size. RAM (Random Access Memory) and ROM (Read Only Memory) are examples of primary storage.

2. Secondary Memory: Secondary storage is a memory that is stored external to the computer. It is mainly used for the permanent and long-term storage of programs and data. Hard Disks, CDs, DVDs, Pen/Flash drives, SSD, etc, are examples of secondary storage.

3. Tertiary Memory: Tertiary Memory is a type of Memory that is rarely used in personal computers and due to this, tertiary memory is not considered to be an important one. Tertiary memory works automatically without human intervention.

Types of Computer Storage Devices

Now we will discuss different types of storage devices available in the market. These storage devices have their own specification and use. Some of the commonly used storage devices are:

1. Primary Storage Devices
2. Magnetic Storage Devices
3. Flash memory Devices
4. Optical Storage Devices
5. Cloud and Virtual Storage

1. Primary Storage Devices

RAM: It stands for Random Access Memory. It is used to store information that is used immediately or we can say that it is a temporary memory. Computers bring the software installed on a hard disk to RAM to process it and to be used by the user.

Once the computer is turned off, the data is deleted. With the help of RAM, computers can perform multiple tasks like loading applications, browsing the web, editing a spreadsheet, experiencing the newest game, etc. It allows you to modify quickly among these tasks, remembering where you're in one task once you switch to a different task. It is also used to load and run applications, like your spreadsheet program, answer commands, like all edits you made within the spreadsheet, or toggle between multiple programs, like once you left the spreadsheet to see the email. Memory is nearly always actively employed by your computer. It ranges from 1GB – 32GB/64GB depending upon the specifications. There are different types of RAM, and although they all serve the same purpose, the most common ones are :

- **SRAM:** It stands for Static Random Access Memory. It consists of circuits that retain stored information as long as the power supply is on. It is also known as volatile memory. It is used to build Cache memory. The access time of SRAM is lower and it is much faster as compared to DRAM but in terms of cost, it is costly as compared to DRAM.
- **DRAM:** It stands for Dynamic Random Access Memory. It is used to store binary bits in the form of electrical charges that are applied to capacitors. The access time of DRAM is slower as compared to SRAM but it is cheaper than SRAM and has a high packaging density.
- **SDRAM:** It stands for Synchronous Dynamic Random Access Memory. It is faster than DRAM. It is widely used in computers and others. After SDRAM was introduced, the upgraded version of double data rate RAM, i.e., DDR1, DDR2, DDR3, and DDR4 was entered into the market and widely used in home/office desktops and laptops.

ROM: It stands for Read-Only Memory. The data written or stored in these devices are non-volatile, i.e, once the data is stored in the memory cannot be modified or deleted. The memory from which will only read but cannot write it. This type of memory is non-volatile. The information is stored permanently during manufacture only once.

ROM stores instructions that are used to start a computer. This operation is referred to as bootstrap. It is also used in other electronic items like washers and microwaves. ROM chips can only store a few megabytes (MB) of data, which ranges between 4 and 8 MB per ROM chip. There are two types of ROM:

- **PROM:** PROM is Programmable Read-Only Memory. These are ROMs that can be programmed. A special PROM programmer is employed to enter the program on the PROM. Once the chip has been programmed, information on the PROM can't be altered. PROM is non-volatile that is data is not lost when power is switched off.

- **EPROM:** Another sort of memory is the Erasable Programmable Read-Only Memory. It is possible to erase the info which has been previously stored on an EPROM and write new data onto the chip.

Difference Between Hard Drive And RAM

Hard Drive	RAM
Non-volatile storage device for long-term data storage	Volatile memory used for temporary storage
Non-volatile (data retained when power is off)	Volatile (data lost when power is off)
Slower access times (HDD slower than SSD)	High-speed access
Typically larger (several TB)	Typically smaller (several GB)

ROM vs RAM

ROM	RAM
Non-volatile memory used for permanent storage	Volatile memory used for temporary storage
Generally slower than RAM	High-speed access
Primarily read-only	Read and write operations
ROM generally holds only megabytes of storage	RAM can store in gigabytes
Data accessible is not easy	Data accessible is easy
Cheaper than RAM	High cost as compared to ROM
Used for the permanent storage of data	Used for temporary storage of data

2. Magnetic Storage Devices

- **Floppy Disk:** Floppy Disk is also known as a floppy diskette. It is generally used on a personal computer to store data externally. A Floppy disk is made up of a plastic cartridge and secured with a protective case. Nowadays floppy disk is replaced by new and effective storage devices like USB, etc.
- **Hard Disk:** Hard Disk is a storage device (HDD) that stores and retrieves data using magnetic storage. It is a non-volatile storage device that can be modified or deleted n number of times without any problem. Most computers and laptops have HDDs as their secondary storage device. It is actually a set of stacked disks, just like

phonograph records. In every hard disk, the data is recorded electromagnetically in concentric circles or we can say track present on the hard disk, and with the help of a head just like a phonograph arm (but fixed in a position) to read the information present on the track. The read-write speed of HDDs is not so fast but decent. It ranges from a few GBs to a few and more TB.

- **Magnetic Card:** It is a card in which data is stored by modifying or rearranging the magnetism of tiny iron-based magnetic particles present on the band of the card. It is also known as a swipe card. It is used like a passcode (to enter the house or hotel room), credit card, identity card, etc.
- **Tape Cassette:** It is also known as a music cassette. It is a rectangular flat container in which the data is stored in an analog magnetic tape. It is generally used to store audio recordings.
- **SuperDisk:** It is also called LS-240 and LS-120. It is introduced by Imation Corporation and it is popular with OEM computers. It can store data up to 240 MB.

3. Flash Memory Devices

It is a cheaper and more portable storage device. It is the most commonly used device to store data because it is more reliable and efficient as compared to other storage devices. Some of the commonly used flash memory devices are:

- **Pen Drive:** It is also known as a USB flash drive that includes flash memory with an integrated USB interface. We can directly connect these devices to our computers and laptops and read/write data into them in a much faster and more efficient way. These devices are very portable. It ranges from 1GB to 256GB generally.
- **SSD:** It stands for Solid State Drive, a mass storage device like HDD. It is more durable because it does not contain optical disks inside like hard disks. It needs less power as compared to hard disks, is lightweight, and has 10x faster read and write speed as compared to hard disks. But, these are costly as well. While SSDs serve an equivalent function as hard drives, their internal components are much different. Unlike hard drives, SSDs don't have any moving parts and thus they're called solid-state drives. Instead of storing data on magnetic platters, SSDs store data using non-volatile storage. Since SSDs haven't any moving parts, they do not need to "spin up". It ranges from 150GB to a few more TB.
- **SD Card:** It is known as a Secure Digital Card. It is generally used with electronic devices like phones, digital cameras, etc. to store larger data. It is portable and the size of the SD card is also small so that it can easily fit into electronic devices. It is available in different sizes like 2GB, 4GB, 8GB, etc.
- **Memory Card:** It is generally used in digital cameras, printers, game consoles, etc. It is also used to store large amounts of data and is available in different sizes. To run a memory card on a computer you require a separate memory card reader.

- **Multimedia Card:** It is also known as MMC. It is an integrated circuit that is generally used in-car radios, digital cameras, etc. It is an external device to store data/information.

4. Optical Storage Devices

Optical Storage Devices is also a secondary storage device. It is a removable storage device. Following are some optical storage devices:

CD: It is known as Compact Disc. It contains tracks and sectors on its surface to store data. It is made up of polycarbonate plastic and is circular in shape. CDs can store data up to 700MB. It is of two types:

CD-R: It stands for Compact Disc read-only. In this type of CD, once the data is written it can not be erased. It is read-only.

CD-RW: It stands for Compact Disc Read Write. In this type of CD, you can easily write or erase data multiple times.

DVD: It is known as Digital Versatile Disc. DVDs are circular flat optical discs used to store data. It comes in two different sizes one is 4.7GB single-layer discs and another one is 8.5GB double-layer discs. DVDs look like CDs but the storage capacity of DVDs is more than as compared to CDs. It is of two types:

DVD-R: It stands for Digital Versatile Disc read-only. In this type of DVD, once the data is written can not be erased. It is read-only. It is generally used to write movies, etc.

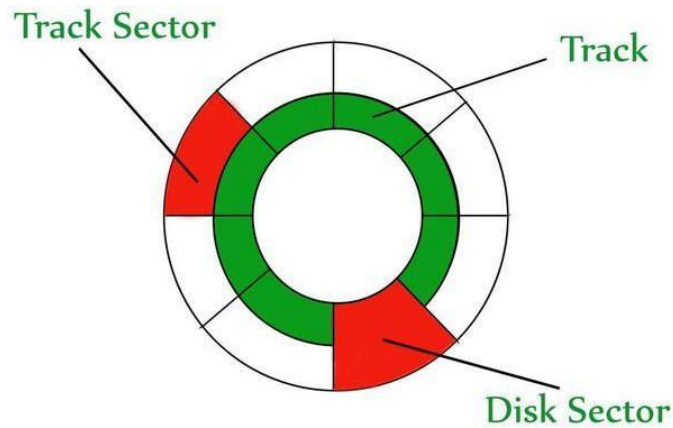
DVD-RW: It stands for Digital Versatile Disc Read Write. In this type of DVD, you can easily write or erase data multiple times.

- **Blu-ray Disc:** It is just like CD and DVD but the storage capacity of blu ray is up to 25GB. To run a Blu-ray disc you need a separate Blu-ray reader. This Blu-ray technology is used to read a disc from a blue-violet laser due to which the information is stored in greater density with a longer wavelength.

5. Hard Disk Drive

The disk is divided into tracks. Each track is further divided into sectors. The point to be noted here is that the outer tracks are bigger than the inner tracks but they contain the same number of sectors and have equal storage capacity. This is because the storage density is high in sectors of the inner tracks whereas the bits are sparsely arranged in sectors of the outer tracks. Some space in every sector is used for formatting. So, the actual capacity of a sector is less than the given capacity. The read-write (R-W) head moves over the rotating hard disk. It is this Read-Write head that performs all the read and write operations on the disk and hence, the position of the R-W head is a major concern. To perform a read or write operation on a memory location, we need to place the R-W head over that position.

Hard disk drives (HDD) serve as secondary storage, providing a non-volatile medium for data.



1. **Seek time** – The time taken by the R-W head to reach the desired track from its current position.
2. **Rotational latency** – Time is taken by the sector to come under the R-W head.
3. **Data transfer time** – Time is taken to transfer the required amount of data. It depends upon the rotational speed.
4. **Controller time** – The processing time taken by the controller.
5. **Average Access time** – seek time + Average Rotational latency + data transfer time + controller time.

Note: Average Rotational latency is mostly $1/2 \times (\text{Rotational latency})$.

4.4 DATA READ & WRITE IN SERVER STORAGE

What is a Server?

A Server is a program or a device that provides functionality for called clients which are other programs or devices. This architecture is called the client-server model.

A single overall computation is distributed across multiple processes or devices. Servers can provide various functionalities called services. These services include sharing data or resources among multiple clients or performing computations for a client. Multiple clients can be served by a single server, and a single client can use multiple servers.

- Server storage refers to storage on a remote server. Data read and write operations on server storage are typically performed using network protocols such as FTP, SSH, or HTTP. These protocols allow users to remotely access and transfer files to and from the server. Additionally, programming languages such as PHP or Python can be used to perform server-side data read and write operations.
- Server storage refers to storage on a remote server, which is typically owned and managed by a business or organization.

- Server storage allows users to store and access data from multiple devices and locations, using network protocols such as FTP, SSH, or HTTP.
- Server storage can be accessed and managed using client software or web interfaces, provided by the server owner.
- The data stored in server storage is typically organized into directories and subdirectories, with access controls and permissions set by the server owner.
- Server storage has several advantages over other storage options, such as local storage and cloud storage. These advantages include:
- Centralization: Server storage allows users to centralize their data storage and management, with easy access and sharing of files and documents across multiple devices and locations. Security: Server storage offers better security than local storage, with access controls and permissions set by the server owner, as well as backups and recovery mechanisms in case of data loss or corruption.
- Collaboration: Server storage allows multiple users to access and work on the same files and documents, with versioning and access controls to ensure security and traceability.
- Customization: Server storage can be customized and configured to meet the specific needs and requirements of the users, with various software and hardware options available for different use cases.
- However, server storage also has some limitations and concerns that users should be aware of. These include potential network issues, such as slow transfer speeds or connectivity problems, as well as potential hardware failures or security risks. Users should carefully evaluate their storage needs and requirements, and choose a server owner that offers the right balance of features, performance, and security.

Types of Servers

Here are the common types of servers based on their functionality:

1. **Web Server:**
 - Hosts websites and delivers web pages to clients via HTTP/HTTPS protocols.
 - Example software: Apache, Nginx, Microsoft IIS.
2. **Example:** When you visit a website, a web server processes your request and sends the site's content to your browser.
3. **Database Server:**
 - Stores, manages, and provides access to databases.
 - Example software: MySQL, PostgreSQL, SQL Server, Oracle Database.
4. **Example:** A banking app retrieves user account details from a database server.

5. File Server:

- Stores and manages files, allowing clients to upload, download, and share them.
- Example: Network Attached Storage (NAS).

6. Example: A shared drive in an office that stores documents and media.

7. Application Server:

- Hosts and runs specific applications for clients, typically handling business logic.
- Example software: JBoss, WebSphere, Apache Tomcat.

8. Example: An e-commerce platform where the application server processes orders.

9. Mail Server:

- Handles email sending, receiving, and storage.
- Example software: Microsoft Exchange, Postfix.

10. Example: Your email app communicates with a mail server to send and receive messages.

11. Proxy Server:

- Acts as an intermediary between clients and other servers, often for security, caching, or filtering.
- Example: Squid Proxy, HAProxy.

12. Example: A proxy server blocks certain websites on a school network.

13. DNS Server:

- Translates domain names (e.g., `www.google.com`) into IP addresses that computers can use.
- Example software: BIND, Cloudflare DNS.

14. Example: Resolving the name "`www.example.com`" to its IP address.

15. Game Server:

- Hosts multiplayer games and facilitates communication between players.
- Example: Minecraft Server, Steam Game Server.

16. Virtual Server:

- A virtualized instance of a physical server, created using virtualization technologies.
- Example software: VMware, Hyper-V.

17. **Example:** Cloud computing services like AWS, Azure, or Google Cloud use virtual servers.

18. **FTP Server:**

- Transfers files between systems using the File Transfer Protocol (FTP).
- Example software: FileZilla Server.

Data stored on a server is managed through a combination of hardware and software systems. These systems ensure that data is saved, retrieved, and organized in a way that supports fast access, security, and efficient management. Let's break down how data is stored on a server:

1. Data Storage Architecture

The way data is stored on a server can be divided into several layers, starting from the physical hardware (disk drives) to the file systems and storage management software.

2. Physical Storage (Hard Drives, SSDs)

At the most basic level, data is stored on physical storage devices attached to the server, such as:

- **Hard Disk Drives (HDDs):** Magnetic storage devices that are slower but offer higher capacity at lower costs.
- **Solid-State Drives (SSDs):** Faster, electronic-based storage devices that provide better performance but are typically more expensive per gigabyte than HDDs.

The physical drives are often grouped into **storage arrays** or **RAID configurations** for redundancy, performance, or both.

3. RAID (Redundant Array of Independent Disks)

RAID is a technology that combines multiple physical drives into a single virtual storage unit to increase performance and/or provide data redundancy.

- **RAID 0 (Striping):** Increases speed by splitting data across multiple drives but offers no redundancy.
- **RAID 1 (Mirroring):** Copies data to two or more drives, providing redundancy.
- **RAID 5 (Striping with Parity):** Combines striping and redundancy (requires at least three drives).
- **RAID 10 (Striped Mirroring):** Combines RAID 1 and RAID 0 for both performance and redundancy.

4. File System

Once the physical storage is set up, the server uses a **file system** to organize and manage data on the drives. The file system handles how data is stored, retrieved, and organized in directories and files.

Common File Systems:

- **NTFS (New Technology File System):** Used primarily in Windows servers.
- **ext4** (Fourth Extended File System): Common in Linux-based servers.
- **XFS:** A high-performance file system commonly used in enterprise environments.
- **FAT32/exFAT:** Older file systems, less common in server environments but used for smaller applications.

5. Data Redundancy and Backup

To prevent data loss, servers often employ redundancy and backup strategies:

- **RAID Configurations:** Protect against drive failure.
- **Backups:** Regular backups (full, incremental, differential) are taken to ensure data can be restored in case of failure.
- **Replication:** Some servers use data replication, where data is copied to another server or storage system, providing an additional layer of protection.

6. Security Considerations in Data Storage

Ensuring that data is securely stored on a server is crucial:

- **Encryption:** Data is often encrypted both at rest (on disk) and in transit (while being transferred over networks) to prevent unauthorized access.
- **Access Control:** Implementing strict user roles and permissions to control who can access, modify, or delete the data.
- **Auditing:** Keeping track of who accessed or modified data for compliance and security purposes.

Characteristics of a Server

- **High Performance:** Servers are designed to handle large workloads and process multiple requests simultaneously.
- **Reliability:** Servers often use redundant hardware and failover mechanisms to minimize downtime.
- **Always On:** Servers typically run 24/7 to ensure uninterrupted service availability.
- **Networked:** Servers operate in networks to provide resources to multiple clients.

Server vs. Client

Aspect	Server	Client
Role	Provides resources or services	Requests resources or services
Examples	Web server, database server	Web browser, email app
Capacity	High (handles multiple requests)	Lower (handles individual tasks)

Applications of Servers

- Hosting websites or applications.
- Managing company data and internal networks.
- Running enterprise applications (e.g., ERP, CRM).
- Storing files or backups.
- Processing and storing user requests in cloud computing.

4.5 DATA READ & WRITE IN CLOUD STORAGE**1. What is Cloud Storage?**

Cloud Computing in general is termed as a different service through the Internet. It has various resources which include tools and applications like data storage, databases, servers, networking, etc. It has applications, platforms, and infrastructure which is surrounded by servers, laptops, desktops, phones, and tablets.

In Cloud Computing, Cloud storage is a virtual locker where we can remotely stash any data. When we upload a file to a cloud-based server like Google Drive, OneDrive, or iCloud that file gets copied over the Internet into a data server that is a cloud-based actual physical space where companies store files on multiple hard drives. Most companies have hundreds of these servers known as ‘server farms’ spanning across multiple locations. So, if our data gets somehow lost we will not lose our data because it will be backed up by another location. This is known as redundancy which keeps our data safe from being lost.

1.1 Features of Cloud Storage System:

The key features of cloud computing are as follows.

- It has a greater availability of resources.
- Easy maintenance is one of the key benefits of using Cloud computing.
- Cloud computing has a Large Network Access.
- It has an automatic system.

- Security is one of the major components and using cloud computing you can secure all over the networks.

1.2 Storage Systems in the Cloud

There are 3 types of storage systems in the Cloud as follows.

- Block-Based Storage System
- File-Based Storage System
- Object-Based Storage System

Let's discuss it one by one as follows.

1. Block-Based Storage System –

- Hard drives are block-based storage systems. Your operating system like Windows or Linux actually sees a hard disk drive. So, it sees a drive on which you can create a volume, and then you can partition that volume and format them.
- For example, If a system has 1000 GB of volume, then we can partition it into 800 GB and 200 GB for local C and local D drives respectively.
- Remember with a block-based storage system, your computer would see a drive, and then you can create volumes and partitions.

2. File-Based Storage System –

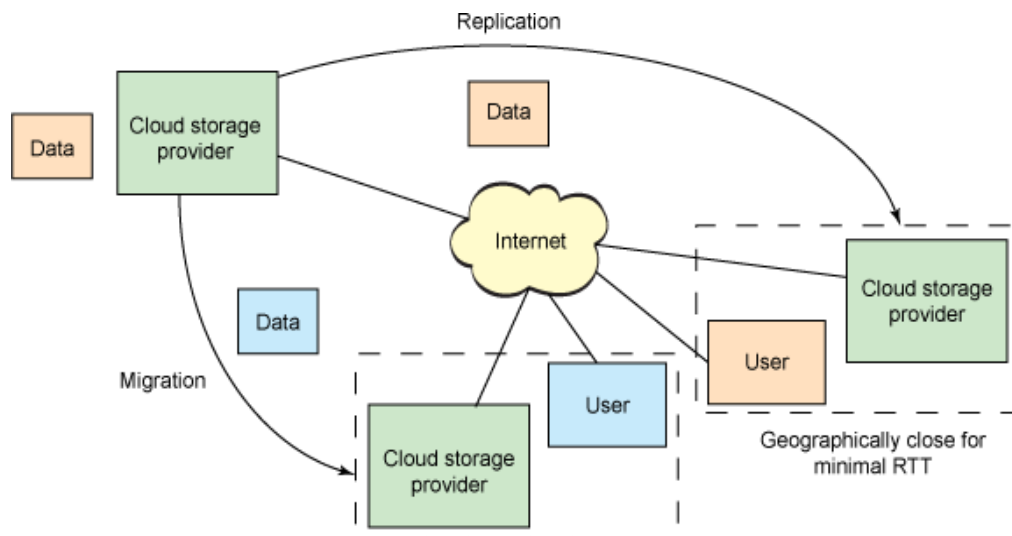
- In this, you are actually connecting through a Network Interface Card (NIC). You are going over a network, and then you can access the network-attached storage server (NAS). NAS devices are file-based storage systems.
- This storage server is another computing device that has another disk in it. It has already created a file system so that it's already formatted its partitions, and it will share its file systems over the network. Here, you can actually map the drive to its network location.
- In this, like the previous one, there is no need to partition and format the volume by the user. It's already done in file-based storage systems. So, the operating system sees a file system that is mapped to a local drive letter.

3. Object-Based Storage System –

- In this, a user uploads objects using a web browser and uploads an object to a container i.e., Object Storage Container. This uses the HTTP Protocols with the rest of the APIs (for example: GET, PUT, POST, SELECT, DELETE).
- For example, when you connect to any website, you need to download some images, text, or anything that the website contains. For that, it is a code HTTP GET request. If you want to review any product then you can use PUT and POST requests.

- Also, there is no hierarchy of objects in the container. Every file is on the same level in an Object-Based storage system.

1.3 Cloud Storage Architecture



Cloud Storage architecture flow is as follows:

- The Cloud Storage Architecture consists of several distributed resources, but still functions as one, either in a cloud architecture of federated or cooperative storage.
- Durable through the manufacture of copies of versions.
- Ultimately, it is usually compatible with data replication advantages.
- Companies just need to pay for the storage they actually use, normally an average of a month's consumption. This does not indicate that cloud storage is less costly, but rather that operating costs are incurred rather than capital expenses.
- Cloud storage companies can cut their energy usage by up to 70 percent, making them a greener company.
- Storage and data security is inherent in the architecture of object storage
- The additional infrastructure, effort, and expense to incorporate accessibility and security can be removed depending on the application.
- Tasks for storage management, such as the procurement of additional storage space, are offloaded to the service provider's obligation.
- It provides users with immediate access to a wide variety of tools and software housed in another organization's infrastructure through a web service interface.
- Very few backups servers are located in different locations across the globe, cloud storage may be used as a natural disaster-proof backup.
- With the WebDAV protocol, cloud storage can be mapped as a local drive

2. Types of Cloud Storage

There are several types of cloud storage based on how they are used and managed:

Public Cloud Storage

- **Definition:** Cloud storage provided by third-party providers like Amazon Web Services (AWS), Google Cloud, or Microsoft Azure. These services are shared with other customers.
- **Examples:**
 - **Amazon S3 (AWS)**
 - **Google Cloud Storage**
 - **Microsoft Azure Blob Storage**
- **Benefits:**
 - Cost-effective and easy to scale.
 - Maintenance, security, and updates are handled by the provider.
- **Use Cases:**
 - Websites, data backup, software applications, and large-scale data storage.

Private Cloud Storage

- **Definition:** Cloud storage that is dedicated to a single organization, often hosted on-premises or in a third-party data center but not shared with other customers.
- **Examples:**
 - Private cloud solutions hosted on VMware or OpenStack.
- **Benefits:**
 - Increased control over security and performance.
 - Customizable to the organization's specific needs.
- **Use Cases:**
 - Organizations with strict compliance requirements or those needing full control over their infrastructure.

Hybrid Cloud Storage

- **Definition:** A combination of public and private cloud storage that allows data and applications to be shared between them, creating a more flexible and optimized environment.

- **Examples:**
 - Using public cloud for less sensitive data and private cloud for more sensitive data.
- **Benefits:**
 - Flexibility to store data based on sensitivity or usage requirements.
 - Ability to scale easily with public cloud storage when needed while keeping critical data secure in private storage.
- **Use Cases:**
 - Enterprises looking to optimize costs while maintaining control over sensitive data.

Type of Cloud Storage	Description	Examples	Use Cases	Benefits
Public Cloud Storage	Storage provided by third-party vendors, shared across multiple users.	Amazon S3, Google Cloud Storage, Azure	Storing backups, media files, and hosting applications.	Cost-effective, scalable, maintenance-free.
Private Cloud Storage	Dedicated storage environment for a single organization, often on-premises.	VMware, OpenStack	Enterprises with strict security or compliance requirements.	High control, customizable, secure.
Hybrid Cloud Storage	Combines public and private cloud storage to balance security and scalability.	AWS Hybrid Cloud, Azure Arc	Storing sensitive data privately while using public cloud for less-critical workloads.	Flexible, cost-optimized, scalable.
Community Cloud Storage	Shared storage by organizations with similar goals or compliance needs.	Custom solutions in healthcare, research	Collaborative projects across institutions with specific regulatory requirements.	Cost sharing, meets specialized compliance n

3. How Cloud Storage Works

1. **Uploading Data:** Users or applications upload files and data to the cloud storage provider using a client (browser-based interface, application, or API). Data is stored in data centers managed by the provider.
2. **Storage Management:** The cloud provider manages all aspects of the infrastructure, including data redundancy, backups, encryption, and scaling. Users do not need to worry about hardware or physical maintenance.
3. **Data Access:** Data is accessed through the internet via the cloud provider's API, web interface, or through integrated applications (e.g., Google Drive, Dropbox, or AWS S3 SDKs).

4. **Redundancy and Backup:** Cloud storage providers often replicate data across multiple locations to ensure high availability and prevent data loss.

4. Benefits of Cloud Storage

a. Scalability

Cloud storage is highly scalable. As your data storage needs grow, you can increase your capacity without the need to purchase or install additional hardware. This flexibility is particularly useful for businesses that experience fluctuating data requirements.

b. Accessibility

Cloud storage allows you to access your data from virtually any device with internet access. This means you can work remotely, collaborate with others, and manage your data from multiple devices (laptops, smartphones, tablets, etc.).

c. Cost-Effective

With cloud storage, you don't need to invest in expensive hardware and infrastructure. Instead, you pay for the storage you use, which can be more cost-efficient for businesses and individuals.

d. Data Redundancy and Reliability

Cloud providers replicate data across multiple locations to ensure redundancy. This means that even if one data center fails, your data remains accessible from other locations, reducing the risk of data loss.

e. Security

Cloud providers offer built-in security measures, such as encryption (data at rest and in transit), access controls, and compliance with industry standards (e.g., GDPR, HIPAA, PCI DSS). This is especially important for organizations storing sensitive or critical data.

f. Disaster Recovery

Cloud storage typically includes disaster recovery capabilities, meaning that your data can be restored quickly in the event of a disaster, like hardware failure or data corruption. This reduces downtime and helps maintain business continuity.

5. Cloud Storage Providers

Here are some of the leading cloud storage providers:

- **Amazon Web Services (AWS):**
 - **Service:** Amazon S3 (Simple Storage Service)
 - **Use:** Object storage for storing large amounts of unstructured data like media files, backups, and web content.

- **Google Cloud:**
 - **Service:** Google Cloud Storage
 - **Use:** Provides high-performance object storage with global distribution, suitable for big data and machine learning applications.
- **Microsoft Azure:**
 - **Service:** Azure Blob Storage
 - **Use:** Scalable object storage for unstructured data, commonly used for app data, backups, and archival storage.
- **Dropbox:**
 - **Service:** Dropbox
 - **Use:** Cloud storage for personal and business files, with features for collaboration and file sharing.
- **Box:**
 - **Service:** Box
 - **Use:** Enterprise-level cloud storage with a focus on collaboration, compliance, and security.
- **Apple iCloud:**
 - **Service:** iCloud
 - **Use:** Cloud storage for personal use, integrated with Apple devices for backing up photos, documents, and apps.
- **pCloud:**
 - **Service:** pCloud
 - **Use:** Secure cloud storage with file-sharing capabilities and options for file encryption.

4.6 Database Query Methods (DDL,DML)

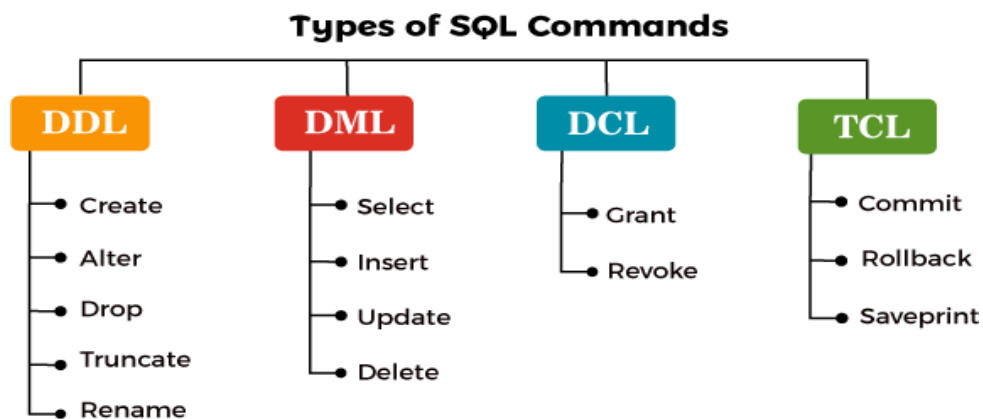
SQL commands are essential for managing databases effectively. These commands are divided into categories such as Data Definition Language (DDL), Data Manipulation Language (DML), Data Control Language (DCL), Data Query Language (DQL), and Transaction Control Language (TCL).

What are SQL Commands?

SQL Commands are like instructions to a table. It is used to interact with the database with some operations. It is also used to perform specific tasks, functions, and queries of data. SQL can perform various tasks like creating a table, adding data to tables, dropping the table,

modifying the table, and setting permission for users. SQL Commands are mainly categorized into five categories:

- DDL – Data Definition Language
- DQL – Data Query Language
- DML – Data Manipulation Language
- DCL – Data Control Language
- TCL – Transaction Control Language



1. What is Data Definition Language (DDL)?

Data Definition Language (DDL) is a type of syntax used for creating, modifying, and deleting database objects, such as tables, indices, views, and sequences. It is similar to a computer programming language that is specifically designed for defining data structures, particularly database schemas. DDL is a subset of the Structured Query Language (SQL).

- Database Objects: These are the entities that are created, modified, or deleted using DDL. They include tables, indices, views, and sequences.
- Database Schemas: A schema is a blueprint of a database that outlines the way data is organized and how the relations among them are associated. It is defined using DDL.
- Structured Query Language (SQL): This is the standard language for managing and manipulating databases. DDL is a part of SQL that deals with database schemas and descriptions of how data should reside in the database.

How are DDL statements used in databases?

DDL statements are used to describe, comment on, and label database objects. They can also be used to impose or drop certain constraints on tables, such as UNIQUE, PRIMARY, FOREIGN KEY, CHECK. Additionally, DDL commands can be used to remove, add, or modify tables within a database.

- **Constraints:** These are the rules enforced on data columns on a table. They are used to limit the type of data that can go into a table. This ensures the accuracy and reliability of the data in the table.
- **DDL Commands:** These are the commands that define a database, including CREATE, ALTER, DROP, TRUNCATE, COMMENT, and RENAME.
- **Modifying Tables:** DDL commands can be used to modify the structure of existing tables. This includes adding or removing columns, changing the type of data stored in a column, and renaming columns.

Common DDL Commands

Command	Description	Syntax
CREATE	Create database or its objects (table, index, function, views, store procedure, and triggers)	CREATE TABLE table_name (column1 data_type, column2 data_type, ...);
DROP	Delete objects from the database	DROP TABLE table_name;
ALTER	Alter the structure of the database	ALTER TABLE table_name ADD COLUMN column_name data_type;
TRUNCATE	Remove all records from a table, including all spaces allocated for the records are removed	TRUNCATE TABLE table_name;
COMMENT	Add comments to the data dictionary	COMMENT 'comment_text' ON TABLE table_name;
RENAME	Rename an object existing in the database	RENAME TABLE old_table_name TO new_table_name;

SQL CREATE TABLE Statement

To create a new table in the database, use the SQL CREATE TABLE statement. A table's structure, including column names, data types, and constraints like NOT NULL, PRIMARY KEY, and CHECK, are defined when it is created in SQL.

The CREATE TABLE command is a crucial tool for database administration because of these limitations, which aid in ensuring data integrity.

Syntax:

To create a table in SQL, use this CREATE TABLE syntax:

```
CREATE table table_name
(
Column1 datatype (size),
column2 datatype (size),
columnN datatype(size)
);
```

- table_name: The name you assign to the new table.
- column1, column2, ... : The names of the columns in the table.
- datatype(size): Defines the data type and size of each column.

Here table_name is name of the table, column is the name of column

SQL CREATE TABLE Example

Let's look at examples of CREATE TABLE commands in SQL and see how to create tables in SQL.

CREATE TABLE in SQL and Insert Data

In this example, we will create a new table and insert data into it.

Let us create a table to store data of Customers, so the table name is Customer, Columns are Name, Country, age, phone, and so on.

```
CREATE TABLE Customer (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    LastName VARCHAR(50),
    Country VARCHAR(50),
    Age INT CHECK (Age >= 0 AND Age <= 99),
    Phone int(10)
);
```

Output:

Customer

CustomerID	CustomerName	LastName	Country	Age	Phone
empty					

CREATE TABLE Customer:

- This creates a new table named Customer.

CustomerID INT PRIMARY KEY:

- CustomerID: An integer column that acts as the **Primary Key**.
- A primary key uniquely identifies each row in the table and cannot be NULL.

CustomerName VARCHAR(50):

- CustomerName: A variable-length string column that can store up to 50 characters for the customer's first name.

LastName VARCHAR(50):

- LastName: Another variable-length string column, similar to CustomerName, for storing the customer's last name.

Country VARCHAR(50):

- Country: A variable-length string column for storing the customer's country.

Age INT CHECK (Age >= 0 AND Age <= 99):

- Age: An integer column for storing the customer's age.
- CHECK (Age >= 0 AND Age <= 99): Ensures the Age value must be between 0 and 99. If a value outside this range is attempted, the database will reject the operation.

Phone INT(10):

- Phone: An integer column intended to store a 10-digit phone number.
- Note: Using INT for phone numbers is problematic because phone numbers often start with leading zeros (which integers cannot store) or include formatting characters (e.g., +, -, spaces). A better approach would be VARCHAR(10).

What is the SQL DROP Command?

In SQL, the DROP command is used to permanently remove an object from a database, such as a table, database, index, or view. When you DROP a table, the table structure and its data are permanently deleted, leaving no trace of the object.

Syntax:

DROP object object_name ;

DROP Command Examples

Let's look at some examples of the DROP statement in SQL

DROP Table**Syntax:**

DROP TABLE table_name;

table_name: Name of the table to be deleted.

DROP database**Syntax:**

DROP DATABASE database_name;

database_name: Name of the database to be deleted.

What is TRUNCATE Command?

The TRUNCATE command is a Data Definition Language (DDL) action that removes all rows from a table but preserves the structure of the table for future use. Although TRUNCATE is similar to the DELETE command (without the WHERE clause), it is much faster because it bypasses certain integrity constraints and locks. It was officially introduced in the SQL:2008 standard.

Syntax:

TRUNCATE TABLE table_name;

Where,

- **table_name:** Name of the table to be truncated.
- **DATABASE name:** student_data

Differences Between DROP and TRUNCATE

The key differences between DROP and TRUNCATE statements are explained in the following table:

DROP	TRUNCATE
In the drop table data and its definition is deleted with their full structure.	It preserves the structure of the table for further use but deletes all the data.
Drop is used to eliminate existing complications and fewer complications in the whole database from the table.	Truncate is used to eliminate the tuples from the table.
Integrity constraints get removed in the DROP command.	Integrity constraint doesn't get removed in the Truncate command.
Since the structure does not exist, the View of the table does not exist in the Drop command.	Since the structure exists, the View of the table exists in the Truncate command.

Drop query frees the table space complications from memory.	This query does not free the table space from memory.
It is slow as there are so many complications compared to the TRUNCATE command.	It is fast as compared to the DROP command as there are fewer complications.

SQL ALTER AND RENAME TABLE STATEMENT

The ALTER TABLE statement in SQL is used to add, remove, or modify columns in an existing table. The ALTER TABLE statement is also used to add and remove various constraints on existing tables.

It allows for structural changes like adding new columns, modifying existing ones, deleting columns, and renaming columns within a table.

Syntax:

To alter/modify the table use the ALTER TABLE syntax:

ALTER TABLE table_name

clause [column_name] [datatype];

Common Use Cases for SQL ALTER TABLE

The **ALTER TABLE** command in SQL is used to modify the structure of an existing table. Below are common use cases for ALTER TABLE, along with examples:

1. Add a New Column

You can add new columns to an existing table.

Use Case: Add a column to store additional information (e.g., adding a "PhoneNumber" column to a "Customers" table).

Syntax:

ALTER TABLE TableName

ADD ColumnName DataType;

Example:

ALTER TABLE Customers

ADD PhoneNumber VARCHAR(15);

2. Drop (Remove) a Column

You can remove a column from the table if it's no longer needed.

Use Case: Remove a redundant or unused column.

Syntax:

```
ALTER TABLE TableName
```

```
DROP COLUMN ColumnName;
```

Example:

```
ALTER TABLE Customers
```

```
DROP COLUMN PhoneNumber;
```

3. Modify the Data Type of a Column

Change the data type or size of an existing column.

Use Case: Increase the size of a string column or change a column's data type.

Syntax:

```
ALTER TABLE TableName
```

```
MODIFY ColumnName NewDataType; -- (MySQL, Oracle)
```

```
ALTER TABLE TableName
```

```
ALTER COLUMN ColumnName NewDataType; -- (SQL Server, PostgreSQL)
```

Example (MySQL):

```
ALTER TABLE Customers
```

```
MODIFY PhoneNumber VARCHAR(20);
```

4. Rename a Column

Rename an existing column in the table.

Use Case: Correct a typo or make the column name more meaningful.

Syntax:

```
ALTER TABLE TableName
```

```
RENAME COLUMN OldColumnName TO NewColumnName; -- (MySQL, PostgreSQL)
```

Example:

```
ALTER TABLE Customers
```

```
RENAME COLUMN PhoneNumber TO ContactNumber;
```

5. Add a Constraint

Add constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, or CHECK to enforce rules on the table.

Use Case: Add a UNIQUE constraint to ensure data integrity.

Syntax:

```
ALTER TABLE TableName
```

```
ADD CONSTRAINT ConstraintName ConstraintType (ColumnName);
```

Example:

```
ALTER TABLE Customers
```

```
ADD CONSTRAINT UC_Email UNIQUE (Email);
```

6. Drop a Constraint

Remove an existing constraint from the table.

Use Case: Remove a CHECK or FOREIGN KEY constraint that's no longer required.

Syntax:

```
ALTER TABLE TableName
```

```
DROP CONSTRAINT ConstraintName; -- (SQL Server, PostgreSQL)
```

```
ALTER TABLE TableName
```

```
DROP INDEX ConstraintName; -- (MySQL for UNIQUE or FOREIGN KEY)
```

Example:

```
ALTER TABLE Customers
```

```
DROP CONSTRAINT UC_Email;
```

7. Rename the Table

Change the name of the table itself.

Use Case: Rename a table to better represent the data it contains.

Syntax:

```
ALTER TABLE OldTableName
```

```
RENAME TO NewTableName; -- (MySQL, PostgreSQL)
```

Example: ALTER TABLE Customers

```
RENAME TO ClientList;
```

2. Data Manipulation Language (DML) in SQL

The **SQL commands** that deal with the **manipulation of data** present in the database belong to **DML** or Data Manipulation Language and this includes most of the **SQL statements**.

It is the component of the SQL statement that controls access to data and to the database. Basically, DCL statements are grouped with DML statements.

Common DML Commands

Command	Description	Syntax
INSERT	Insert data into a table	INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
UPDATE	Update existing data within a table	UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
DELETE	Delete records from a database table	DELETE FROM table_name WHERE condition;
LOCK	Table control concurrency	LOCK TABLE table_name IN lock_mode;
CALL	Call a PL/SQL or JAVA subprogram	CALL procedure_name(arguments);
EXPLAIN PLAN	Describe the access path to data	EXPLAIN PLAN FOR SELECT * FROM table_name;

Generally, DML commands fall into one of four primary categories:

- **INSERT** adds fresh data to a table.
- **UPDATE** Change the data that is already in a table.
- **DELETE** takes a record out of a table.
- **SELECT** Get information out of one or more tables.

Characteristics of DML

It performs interpret-only data queries. It is used in a database schema to recall and manipulate the information. DML It is a dialect which is used to select, insert, delete and update data in a database. Data Manipulation Language (DML) commands are as follows:

SQL INSERT INTO Statement

The INSERT INTO statement is a fundamental **SQL command** used to add new rows of data into a table in a database. It is one of the most commonly used SQL statements for manipulating data and plays a key role in **database management**.

This article will explore the **SQL INSERT INTO** statement in detail, showing how to use it effectively with multiple examples. We'll cover both basic and advanced usage, including inserting individual rows, **multiple rows**, and even copying data between tables.

SQL INSERT INTO Statement

- The **INSERT INTO** statement in **SQL** is used to add new rows of data to a table in a database.
- There are two main ways to use the **INSERT INTO** statement by specifying the columns and values explicitly or by inserting values for all columns without specifying them.

There are two primary syntaxes of **INSERT INTO** statements depending on the requirements. The two syntaxes are:

Syntax 1. Only Values

The first method is to specify only the value of data to be inserted without the column names.

```
INSERT INTO table_name
```

```
VALUES (value1, value2, value);
```

Parameters:

- **table_name:** name of the table.
- **value1, value2:** value of first column, second column,... for the new record

Syntax 2. Column Names And Values Both

In the second method we will specify both the columns which we want to fill and their corresponding values as shown below:

```
INSERT INTO table_name (column1, column2, column3)
```

```
VALUES ( value1, value2, value);
```

Parameters:

- **table_name:** name of the table.
- **column1, column2..:** name of first column, second column.
- **value1, value2, value..:** the values for each specified column of the new record.

Examples of SQL INSERT INTO

For better understanding, let's look at the SQL Server **INSERT** statement with examples.

Let us first create a table named '**Student**'.

```
CREATE DATABASE StudentDB;
```

```
USE StudentDB;
```

```
CREATE TABLE Student (
```

```
    ROLL_NO INT PRIMARY KEY,
```

```

NAME VARCHAR(50),
ADDRESS VARCHAR(100),
PHONE VARCHAR(15),
AGE INT
);

INSERT INTO Student (ROLL_NO, NAME, ADDRESS, PHONE, AGE) VALUES
(1, 'Ram', 'Delhi', 'XXXXXXXXXXXX', 18),
(2, 'Ramesh', 'Gurgaon', 'XXXXXXXXXXXX', 18),
(3, 'Sujit', 'Rohtak', 'XXXXXXXXXXXX', 20),
(4, 'Suresh', 'Rohtak', 'XXXXXXXXXXXX', 18);

```

Created Table:

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	Ram	Delhi	XXXXXXXXXXXX	18
2	RAMESH	GURGAON	XXXXXXXXXXXX	18
3	SUJIT	ROHTAK	XXXXXXXXXXXX	20
4	SURESH	ROHTAK	XXXXXXXXXXXX	18
3	SUJIT	ROHTAK	XXXXXXXXXXXX	20
2	RAMESH	GURGAON	XXXXXXXXXXXX	18

Now, Suppose we want to add values.

Example 1: Inserting Only New Values Using INSERT INTO Example

If we want to insert only values then we use the following query.

Query:

```

INSERT INTO Student
VALUES ('5','HARSH','WEST BENGAL', 'XXXXXXXXXX','19');

```

Output:

The table **Student** will now look like this:

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	Ram	Delhi	XXXXXXXXXXXX	18
2	RAMESH	GURGAON	XXXXXXXXXXXX	18
3	SUJIT	ROHTAK	XXXXXXXXXXXX	20
4	SURESH	Delhi	XXXXXXXXXXXX	18
3	SUJIT	ROHTAK	XXXXXXXXXXXX	20
2	RAMESH	GURGAON	XXXXXXXXXXXX	18
5	HARSH	WEST BENGAL	XXXXXXXXXXXX	19

Example 2: Insert Values to Specific Columns Using INSERT INTO Example

If we want to insert values in the specified columns then we use the following query.

Query:

INSERT INTO Student (ROLL_NO, NAME, Age)

VALUES ('5','PRATIK','19');

Output:

The table **Student** will now look like this:

ROLL_NO	NAME	ADDRESS	PHONE	Age
1	Ram	Delhi	XXXXXXXXXXXX	18
2	RAMESH	GURGAON	XXXXXXXXXXXX	18
3	SUJIT	ROHTAK	XXXXXXXXXXXX	20
4	SURESH	Delhi	XXXXXXXXXXXX	18
3	SUJIT	ROHTAK	XXXXXXXXXXXX	20
2	RAMESH	GURGAON	XXXXXXXXXXXX	18
5	PRATIK	null	null	19

Note: Columns not included in the INSERT statement are filled with default values (typically NULL).

Output:

ROLL_NO	NAME	ADDRESS	PHONE	AGE
1	Ram	Delhi	XXXXXXXXXXXX	18
2	Ramesh	Gurgaon	XXXXXXXXXXXX	18
3	Sujit	Rohtak	XXXXXXXXXXXX	20
4	Suresh	Rohtak	XXXXXXXXXXXX	18
5	Pratik	NULL	NULL	19
6	Amit Kumar	Delhi	XXXXXXXXXXXX	15
7	Gauri Rao	Bangalore	XXXXXXXXXXXX	18
8	Manav Bhatt	New Delhi	XXXXXXXXXXXX	17
9	Riya Kapoor	Udaipur	XXXXXXXXXXXX	10

If a user wants to insert more than 1000 rows, multiple insert statements, bulk insert or derived table must be used.

SQL UPDATE Statement

In SQL, the **UPDATE statement** is used to modify existing records in a table. Whether you need to update a single record or multiple rows at once, SQL provides the necessary functionality to make these changes. The UPDATE statement is essential for maintaining the integrity of your data by allowing you to correct or **modify existing** entries in your database.

In this article, we will cover the fundamentals of the UPDATE statement, including its **syntax**, basic usage, and advanced techniques. We will also explore common pitfalls, optimization tips, and **real-world examples** to ensure you can use the UPDATE statement effectively in your SQL queries.

UPDATE Statement in SQL

The UPDATE statement in SQL is used to update the data of an existing table in the database. We can update single columns as well as multiple columns using the UPDATE statement as per our requirement.

Syntax:

The syntax for SQL UPDATE Statement is :

```
UPDATE table_name SET column1 = value1, column2 = value2,...
WHERE condition;
```

Where,

- **table_name:** name of the table
- **column1:** name of first, second, third column....

- **value1:** new value for first, second, third column....
- **condition:** condition to select the rows for which the

Parameter Explanation

1. **UPDATE:** Command is used to update the column value in the table.
2. **WHERE:** Specifies the condition which we want to implement on the table.

Note: In the above query the **SET** statement is used to set new values to the particular column and the **WHERE** clause is used to select the rows for which the columns are needed to be updated. If we have not used the WHERE clause then the columns in all the rows will be updated. So the WHERE clause is used to choose the particular rows.

How to Use the SQL UPDATE Statement

Let's see the SQL update statement with examples.

First we will create a table, on which we will use the UPDATE Statement. To create the table, write the following query

Query:

```
CREATE TABLE Customer(
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    LastName VARCHAR(50),
    Country VARCHAR(50),
    Age int(2),
    Phone int(10)
);

-- Insert some sample data into the Customers table

INSERT INTO Customer (CustomerID, CustomerName, LastName, Country, Age, Phone)
VALUES (1, 'Shubham', 'Thakur', 'India', '23', 'xxxxxxxxxx'),
       (2, 'Aman ', 'Chopra', 'Australia', '21', 'xxxxxxxxxx'),
       (3, 'Naveen', 'Tulasi', 'Sri lanka', '24', 'xxxxxxxxxx'),
       (4, 'Aditya', 'Arpan', 'Austria', '21', 'xxxxxxxxxx'),
       (5, 'Nishant. Salchichas S.A.', 'Jain', 'Spain', '22', 'xxxxxxxxxx');
```

The created table will look like this:

Customer

CustomerID	CustomerName	LastName	Country	Age	Phone
1	Shubham	Thakur	India	23	xxxxxxxxxx
2	Aman	Chopra	Australia	21	xxxxxxxxxx
3	Naveen	Tulasi	Sri lanka	24	xxxxxxxxxx
4	Aditya	Arpan	Austria	21	xxxxxxxxxx
5	Nishant. Salchichas S.A.	Jain	Spain	22	xxxxxxxxxx

SQL DELETE Statement

The **SQL DELETE** statement is one of the most commonly used **commands** in SQL (Structured Query Language). It allows you to remove one or more rows from the table depending on the situation. Unlike the **DROP statement**, which removes the entire table, the **DELETE statement** removes data (rows) from the table retaining only the table structure, constraints, and schema

In this article, we'll learn the syntax and usage of the SQL DELETE statement with detailed examples, including how to **delete a single record**, delete multiple records, and **delete all records** from a table.

SQL DELETE Statement

The SQL DELETE statement **removes** one or more rows from a database table based on a condition specified in the **WHERE** clause. It's a DML (Data Manipulation Language) operation that modifies the data within the table without altering its structure.

Syntax:

```
DELETE FROM table_name
```

```
WHERE some_condition;
```

Parameter Explanation

- **Some_condition:** condition to choose a particular record.
- **table_name:** name of the table

Note: We can delete single as well as multiple records depending on the condition we provide in the WHERE clause. If we omit the WHERE clause then all of the records will be deleted and the table will be empty.

The sample table is as follows GFG_Employees:

Examples of SQL DELETE Statement

Assume we have created a table named GFG_Employee in SQL which contains the personal details of the Employee including their id, name, email and department etc. as shown below –

```
CREATE TABLE GFG_Employees (
    id INT PRIMARY KEY,
    name VARCHAR (20) ,
    email VARCHAR (25),
    department VARCHAR(20)
);

INSERT INTO GFG_Employees (id, name, email, department) VALUES
(1, 'Jessie', 'jessie23@gmail.com', 'Development'),
(2, 'Praveen', 'praveen_dagger@yahoo.com', 'HR'),
(3, 'Bisa', 'dragonBall@gmail.com', 'Sales'),
(4, 'Rithvik', 'msvv@hotmail.com', 'IT'),
(5, 'Suraj', 'srjsunny@gmail.com', 'Quality Assurance'),
(6, 'Om', 'OmShukla@yahoo.com', 'IT'),
(7, 'Naruto', 'uzumaki@konoha.com', 'Development');

Select * From GFG_Employees
```

Output

id	name	email	department
1	Jessie	jessie23@gmail.com	Development
2	Praveen	praveen_dagger@yahoo.com	HR
3	Bisa	dragonBall@gmail.com	Sales
4	Rithvik	msvv@hotmail.com	IT
5	Suraj	srjsunny@gmail.com	Quality Assurance
6	Om	OmShukla@yahoo.com	IT
7	Naruto	uzumaki@konoha.com	Development

GFG_Employees

Example 1: Deleting Single Record

We can use the **DELETE statement** with a condition to delete a specific row from a table. The WHERE clause ensures only the intended record is removed. We can delete the records named Rithvik by using the below query:

Query:

```
DELETE FROM GFG_Employees WHERE NAME = 'Rithvik';
```

Output:

id	name	email	department
1	Jessie	jessie23@gmail.com	Development
2	Praveen	praveen_dagger@yahoo.com	HR
3	Bisa	dragonBall@gmail.com	Sales
5	Suraj	srjsunny@gmail.com	Quality Assurance
6	Om	OmShukla@yahoo.com	IT
7	Naruto	uzumaki@konoha.com	Development